

Лабораторная работа №4
«Аппроксимация функции методом наименьших квадратов»
Вариант №11

Выполнил:
Студент группы Р3208
Решетников С.Е.

Преподаватели:
Рыбаков С.Д.

Дата сдачи: 1 мая
Санкт-Петербург, 2026

1. Цель работы

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

2. Вычислительная реализация

2.1. Исходные данные

$$y = \frac{5x}{x^4 + 11}$$
$$x \in [-2; 0] \quad h = 0.2$$

2.2. Таблица табулирования

x	-2	-1.8	-1.6	-1.4	-1.2	-1	-0.8	-0.6	-0.4	-0.2	0
y	-0.370	-0.419	-0.456	-0.472	-0.459	-0.417	-0.351	-0.270	-0.181	-0.091	0

2.3. Построение аппроксимаций

Для заданной табличной функции требуется построить:

- линейное приближение вида $y = ax + b$;
- квадратичное приближение вида $y = a_0 + a_1x + a_2x^2$.

2.4. Линейная аппроксимация

Общий вид линейной функции:

$$y = ax + b$$

Коэффициенты a и b находятся из системы нормальных уравнений:

$$\begin{cases} nb + a \sum x_i = \sum y_i \\ b \sum x_i + a \sum x_i^2 = \sum x_i y_i \end{cases}$$
$$\sum x_i = -11 \quad \sum y_i = -3.484 \quad \sum x_i y_i = 4.384 \quad \sum x_i^2 = 15.4$$
$$\begin{cases} 11 \cdot b + a \cdot (-11) = -3.484 \\ b \cdot (-11) + a \cdot (15.4) = 4.384 \end{cases} \Rightarrow \begin{cases} 11 \cdot b + a \cdot (-11) = -3.484 \\ a \cdot 4.4 = 0.9 \end{cases} \Rightarrow \begin{cases} 11 \cdot b = -1.234 \\ a = 0.205 \end{cases} \Rightarrow \begin{cases} b = -0.112 \\ a = 0.205 \end{cases}$$

Получаем приближение:

$$y = 0.205 \cdot x - 0.112$$
$$\text{MSE} : 0.013$$

2.5. Квадратичная аппроксимация

Общий вид квадратичной функции:

$$y = a_0 + a_1x + a_2x^2$$

Коэффициенты находятся из системы:

$$\begin{cases} na_0 + a_1 \sum x_i + a_2 \sum x_i^2 = \sum y_i \\ a_0 \sum x_i + a_1 \sum x_i^2 + a_2 \sum x_i^3 = \sum x_i y_i \\ a_0 \sum x_i^2 + a_1 \sum x_i^3 + a_2 \sum x_i^4 = \sum x_i^2 y_i \end{cases}$$

$$\sum x_i = -11 \quad \sum x_i^2 = 15.4 \quad \sum x_i^3 = -24.2 \quad \sum x_i^4 = 40.533$$

$$\sum y_i = -3.484 \quad \sum x_i y_i = 4.384 \quad \sum x_i^2 y_i = -6.361$$

$$\begin{cases} 11a_0 + a_1 \cdot (-11) + a_2 \cdot 15.4 = -3.484 \\ a_0 \cdot (-11) + a_1 \cdot 15.4 + a_2 \cdot (-24.2) = 4.384 \\ a_0 \cdot 15.4 + a_1 \cdot (-24.2) + a_2 \cdot 40.533 = -6.361 \end{cases} \Leftrightarrow \left(\begin{array}{ccc|c} 11 & -11 & 15.4 & -3.484 \\ -11 & 15.4 & -24.2 & 4.384 \\ 15.4 & -24.2 & 40.533 & -6.361 \end{array} \right)$$

$$\begin{cases} a_0 = 0.026 \\ a_1 = 0.666 \\ a_2 = 0.231 \end{cases}$$

Получаем приближение:

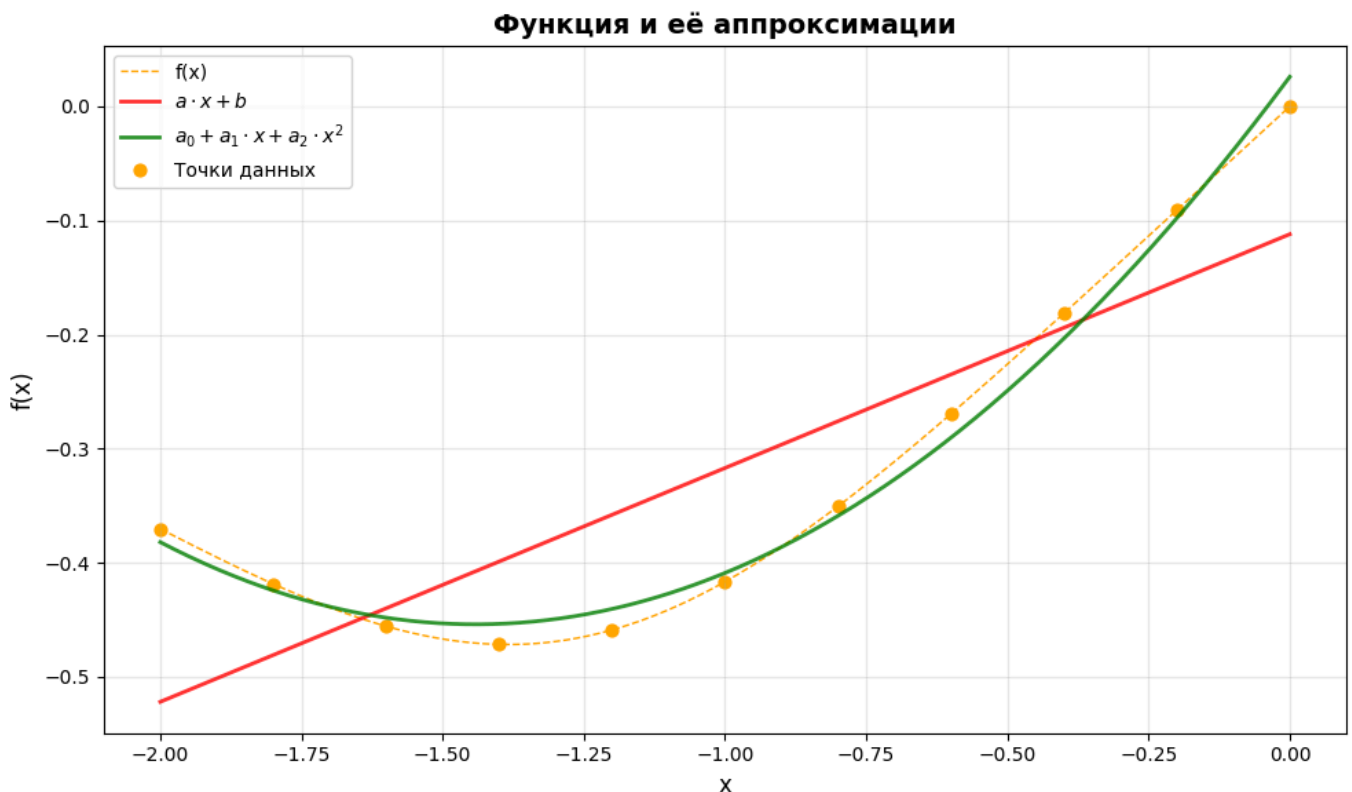
$$y = 0.026 + 0.666x + 0.231x^2$$

$$\text{MSE} : 0.001$$

2.6. Выбор наилучшего приближения

Квадратичное приближение показывает себя лучше

2.7. Графики функции и её приближений



3. Программная реализация

3.1. Частичный листинг программы

```
import numpy as np
from approx.base import ApproximationModel
```

```

class CubicApprox:
    """Кубическая аппроксимация  $y = a_0 + a_1 x + a_2 x^2 + a_3 x^3$ """
    name = "cubic"

    def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
        sum_x = np.sum(x)
        sum_y = np.sum(y)
        sum_x2 = np.sum(x ** 2)
        sum_x3 = np.sum(x ** 3)
        sum_x4 = np.sum(x ** 4)
        sum_x5 = np.sum(x ** 5)
        sum_x6 = np.sum(x ** 6)
        sum_xy = np.sum(x * y)
        sum_x2y = np.sum((x ** 2) * y)
        sum_x3y = np.sum((x ** 3) * y)

        A = np.array([
            [len(x), sum_x, sum_x2, sum_x3],
            [sum_x, sum_x2, sum_x3, sum_x4],
            [sum_x2, sum_x3, sum_x4, sum_x5],
            [sum_x3, sum_x4, sum_x5, sum_x6]
        ], dtype=float)
        B = np.array([sum_y, sum_xy, sum_x2y, sum_x3y])

        try:
            coeffs = np.linalg.solve(A, B)
            a0, a1, a2, a3 = coeffs

            def predict(x_values: np.ndarray) -> np.ndarray:
                return a0 + a1 * x_values + a2 * x_values ** 2 + a3 * x_values ** 3

            return ApproximationModel(
                name=self.name,
                equation=f"y = {a0:.6f} + {a1:.6f}*x + {a2:.6f}*x^2 + {a3:.6f}*x^3",
                coeffs={"a0": float(a0), "a1": float(a1), "a2": float(a2), "a3": float(a3)},
                predict=predict,
            )
        except np.linalg.LinAlgError:
            raise ValueError("Ошибка при вычислении кубической аппроксимации")

import numpy as np
from approx.base import ApproximationModel

class ExponentialApprox:
    """
    Экспоненциальная аппроксимация  $y = a * \exp(bx)$ 
    Преобразуем в линейную:  $\ln(y) = \ln(a) + bx$ 
    """
    name = "exponential"

    def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
        if np.any(y <= 0):
            raise ValueError("Экспоненциальная аппроксимация требует y > 0")

        sum_x = np.sum(x)
        ln_y = np.log(y)
        sum_x2 = np.sum(x ** 2)
        sum_ln_y = np.sum(ln_y)
        sum_x_ln_y = np.sum(x * ln_y)

        try:
            A = np.array([[len(x), sum_x], [sum_x, sum_x2]], dtype=float)
            B = np.array([sum_ln_y, sum_x_ln_y])
            coeffs = np.linalg.solve(A, B)

            ln_a, b = coeffs
            a = np.exp(ln_a)

            def predict(x_values: np.ndarray) -> np.ndarray:
                return a * np.exp(b * x_values)

            return ApproximationModel(
                name=self.name,
                equation=f"y = {a:.6f}*exp({b:.6f}*x)",
                coeffs={"a": float(a), "b": float(b)},
                predict=predict,
            )
        except np.linalg.LinAlgError:
            raise ValueError("Ошибка при вычислении экспоненциальной аппроксимации")

import numpy as np
from approx.base import ApproximationModel

class LinearApprox:
    """Линейная аппроксимация  $y = ax + b$ """
    name = "linear"

```

```

def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
    sum_x = np.sum(x)
    sum_y = np.sum(y)
    sum_x2 = np.sum(x ** 2)
    sum_xy = np.sum(x * y)

    A = np.array([[len(x), sum_x], [sum_x, sum_x2]], dtype=float)
    B = np.array([sum_y, sum_xy])

    try:
        coeffs = np.linalg.solve(A, B)
        b, a = coeffs

    def predict(x_values: np.ndarray) -> np.ndarray:
        return a * x_values + b

    return ApproximationModel(
        name=self.name,
        equation=f"y = {a:.6f}*x + {b:.6f}",
        coeffs={"a": float(a), "b": float(b)},
        predict=predict,
    )
    except np.linalg.LinAlgError:
        raise ValueError("Ошибка при вычислении линейной аппроксимации")

import numpy as np
from approx.base import ApproximationModel

class LogarithmicApprox:
    """
    Логарифмическая аппроксимация  $y = a * \ln(x) + b$ 
    Преобразуем в линейную:  $y = a * \ln(x) + b$ 
    """
    name = "logarithmic"

    def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
        if np.any(x <= 0):
            raise ValueError("Логарифмическая аппроксимация требует x > 0")

        ln_x = np.log(x)
        sum_ln_x = np.sum(ln_x)
        sum_y = np.sum(y)
        sum_ln_x2 = np.sum(ln_x ** 2)
        sum_ln_x_y = np.sum(ln_x * y)

        try:
            A = np.array([[len(x), sum_ln_x], [sum_ln_x, sum_ln_x2]], dtype=float)
            B = np.array([sum_y, sum_ln_x_y])

            coeffs = np.linalg.solve(A, B)
            b, a = coeffs

        def predict(x_values: np.ndarray) -> np.ndarray:
            values = np.asarray(x_values, dtype=float)
            y_approx = np.full_like(values, np.nan)
            mask = values > 0
            y_approx[mask] = a * np.log(values[mask]) + b
            return y_approx

        return ApproximationModel(
            name=self.name,
            equation=f"y = {a:.6f}*ln(x) + {b:.6f}",
            coeffs={"a": float(a), "b": float(b)},
            predict=predict,
        )
        except np.linalg.LinAlgError:
            raise ValueError("Ошибка при вычислении логарифмической аппроксимации")

import numpy as np
from approx.base import ApproximationModel

class PowerApprox:
    """
    Степенная аппроксимация  $y = a * x^b$ 
    Преобразуем в линейную:  $\ln(y) = \ln(a) + b * \ln(x)$ 
    """
    name = "power"

    def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
        if np.any(x <= 0):
            raise ValueError("Степенная аппроксимация требует x > 0")
        if np.any(y <= 0):
            raise ValueError("Степенная аппроксимация требует y > 0")

        ln_x = np.log(x)
        ln_y = np.log(y)
        sum_ln_x = np.sum(ln_x)
        sum_ln_y = np.sum(ln_y)

```

```

sum_ln_x2 = np.sum(ln_x ** 2)
sum_ln_x_ln_y = np.sum(ln_x * ln_y)

try:
    A = np.array([[len(x), sum_ln_x], [sum_ln_x, sum_ln_x2]], dtype=float)
    B = np.array([sum_ln_y, sum_ln_x_ln_y])

    coeffs = np.linalg.solve(A, B)
    ln_a, b = coeffs
    a = np.exp(ln_a)

    def predict(x_values: np.ndarray) -> np.ndarray:
        values = np.asarray(x_values, dtype=float)
        y_approx = np.full_like(values, np.nan)
        mask = values > 0
        y_approx[mask] = a * np.power(values[mask], b)
        return y_approx

    return ApproximationModel(
        name=self.name,
        equation=f"y = {a:.6f}*x^{b:.6f}",
        coeffs={"a": float(a), "b": float(b)},
        predict=predict,
    )
except np.linalg.LinAlgError:
    raise ValueError("Ошибка при вычислении степенной аппроксимации")

import numpy as np
from approx.base import ApproximationModel

class QuadraticApprox:
    """Квадратичная аппроксимация y = a_0 + a_1 x + a_2 x^2"""
    name = "quadratic"

    def fit(self, x: np.ndarray, y: np.ndarray) -> ApproximationModel:
        sum_x = np.sum(x)
        sum_y = np.sum(y)
        sum_x2 = np.sum(x ** 2)
        sum_x3 = np.sum(x ** 3)
        sum_x4 = np.sum(x ** 4)
        sum_xy = np.sum(x * y)
        sum_x2y = np.sum((x ** 2) * y)

        A = np.array([
            [len(x), sum_x, sum_x2],
            [sum_x, sum_x2, sum_x3],
            [sum_x2, sum_x3, sum_x4]
        ], dtype=float)
        B = np.array([sum_y, sum_xy, sum_x2y])

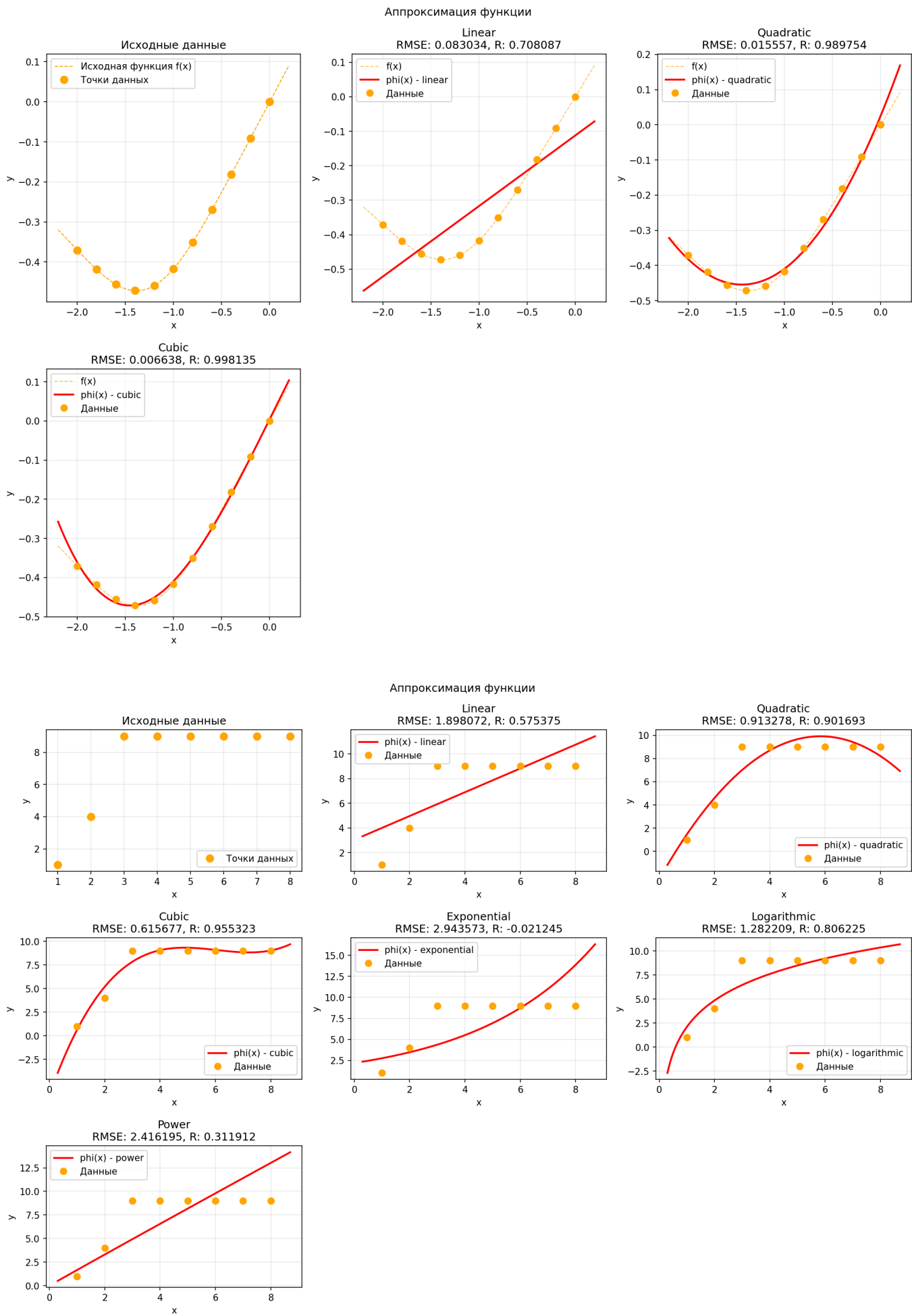
        try:
            coeffs = np.linalg.solve(A, B)
            a0, a1, a2 = coeffs

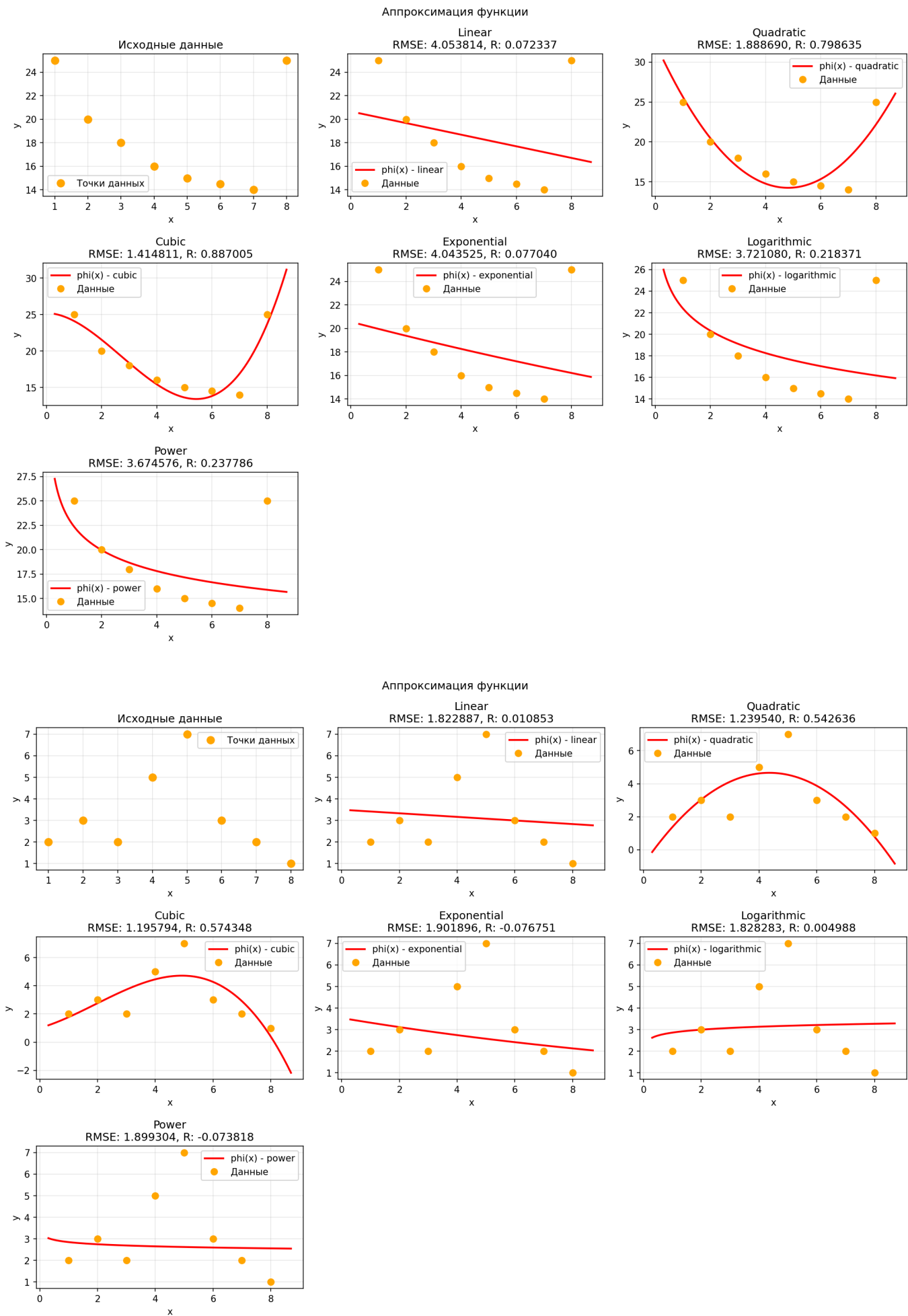
            def predict(x_values: np.ndarray) -> np.ndarray:
                return a0 + a1 * x_values + a2 * x_values ** 2

            return ApproximationModel(
                name=self.name,
                equation=f"y = {a0:.6f} + {a1:.6f}*x + {a2:.6f}*x^2",
                coeffs={"a0": float(a0), "a1": float(a1), "a2": float(a2)},
                predict=predict,
            )
        except np.linalg.LinAlgError:
            raise ValueError("Ошибка при вычислении квадратичной аппроксимации")

```

3.2. Результаты работы





4. Выводы

В ходе лабораторной работы изучен метод наименьших квадратов для аппроксимации таблично заданной функции $y = \frac{5x}{x^4+11}$ на отрезке $[-2; 0]$ с шагом 0.2.

Построены две аппроксимирующие функции:

- линейная: $y = 0.205x - 0.112$ (MSE = 0.013);
- квадратичная: $y = 0.026 + 0.666x + 0.231x^2$ (MSE = 0.001).

Квадратичное приближение оказалось точнее линейного (MSE меньше на порядок), что объясняется нелинейным характером исходной функции.

Разработанная программа позволяет строить линейные, квадратичные, кубические, экспоненциальные, логарифмические и степенные аппроксимации. Программа успешно протестирована на заданных данных.

Цель работы достигнута.